

czkawka / Krokiet 사용법 및 WSL 대체 CLI 도구 워크플로우

1. 사용자

czkawka를 설치하는 방법을 알려줘 gui, cli 모두 대상이며 windows, mac 각각 알려줘

2. Claude

czkawka / Krokiet 설치 방법

czkawka(중복 파일 정리 도구)는 최신 버전(12.0.1 기준)부터 **GTK GUI는 개발이 중단**되었고, 후속 GUI인 **Krokiet**으로 이전할 것을 공식적으로 권장하고 있습니다. 아래는 GUI와 CLI를 Windows/Mac 각각 설치하는 방법입니다.

1. Windows

GUI (Krokiet, 권장)

- [GitHub Releases 페이지](https://github.com/qarmin/czkawka/releases/latest)에서 아래 파일 중 하나를 다운로드합니다.
 - windows_krokiet_on_windows_skia_opengl.exe (기본 추천)
 - 안 될 경우 windows_krokiet_on_linux.exe
- 별도 설치 프로그램 없이 exe 파일을 실행하면 바로 사용 가능합니다(포터블 앱).
- 참고: 일부 Skia 백엔드는 **Microsoft Visual C++ Redistributable** 설치가 필요할 수 있습니다.

GUI (winget으로 설치)

```
winget install Czkawka
```

Chocolatey를 쓰신다면 `choco install czkawka`도 가능합니다.

GUI (구버전 GTK, 참고용)

- windows_czkawka_gui_gtk_412.zip 다운로드 후 압축 해제하여 실행 (더 이상 신규 기능 업데이트는 없음)

CLI

- Releases 페이지에서 CLI 용 파일을 다운로드합니다. Windows 용 CLI 바이너리는 Linux 목록에 있는 것과 달리 별도 표기가 있으니, 페이지에서 windows + cli 가 포함된 자산을 찾아 받으시면 됩니다(예: windows_czkawka_cli...exe).
- 다운로드한 exe 를 원하는 폴더에 두고, 터미널(PowerShell/cmd)에서 실행 파일 경로로 명령어를 입력해 사용합니다.

3. 예시: `czkawka_cli.exe dup -d C:\Users\사용자명\Downloads`
4. HEIF/RAW 등 확장 이미지 포맷 지원이 필요하다면 `choco install ffmpeg` 등을 추가로 설치해야 하며, Windows에서는 heif/libraw 기능이 사전 빌드 바이너리에 포함되어 있지 않아 설정이 까다로운 편입니다.

2. macOS

GUI (Krokiet, 권장) — Homebrew

```
brew install --cask ...
```

공식 Homebrew formula는 아직 CLI용이 중심이라, GUI는 아래 직접 다운로드 방식이 더 확실합니다.

GUI (직접 다운로드)

1. [Releases 페이지](https://github.com/qarmin/czkawka/releases/latest)에서 Mac 아키텍처에 맞는 파일을 선택합니다.
 - Apple Silicon(M1/M2/M3 등): `mac_krokiet_skia_vulkan_arm64` 또는 `mac_czkawka_gui_arm64`
 - Intel Mac: `mac_krokiet_skia_vulkan_x86_64` 또는 `mac_czkawka_gui_x86_64`

1. 터미널에서 다음과 같이 실행 권한을 부여하고 실행합니다.

```
cd "$HOME/Downloads"
chmod +x mac_krokiet_skia_vulkan_arm64
./mac_krokiet_skia_vulkan_arm64
mv mac_krokiet_skia_vulkan_arm64 /Applications/
```

1. Gatekeeper 가 서명되지 않은 앱을 차단할 수 있으니, 처음 실행 시 우클릭 → "열기"를 선택해야 합니다.

GUI — Homebrew (구버전 czkawka, 참고용)

```
brew install czkawka
```

GUI — MacPorts

```
sudo port install czkawka
```

CLI

- Homebrew 로 설치 시 `czkawka_cli` 바이너리도 함께 딸려오는 경우가 많으나, 최신 CLI 만 원하시면 Releases 에서 macOS 용 `mac_czkawka_cli_...` 자산을 직접 받아 위와 동일하게 `chmod +x` 후 실행하시면 됩니다.
- HEIF/RAW/AVIF 등 확장 포맷을 쓰려면 다음을 추가로 설치합니다.

```
brew install ffmpeg libraw libheif libavif dav1d
```

참고 사항

- **czkawka(GTK GUI)**는 12.0 버전을 마지막으로 신규 바이너리가 제공되지 않으며, 개발자도 신규 사용자에게 **Krokiet** 사용을 권장하고 있습니다.
- CLI 는 Windows 10 이상, macOS 10.15 이상에서 별도 종속성 없이 바로 실행 가능합니다.
- 정확한 최신 자산 목록은 GitHub Releases 페이지에서 매번 확인하시는 것이 가장 안전합니다: <https://github.com/qarmin/czkawka/releases/latest>

3. 사용자

krokiet 의 사용법을 알려줘. cli, gui 버전 각각 자세히 알려줘

4. Claude

공식 문서를 좀 더 자세히 확인하겠습니다.

충분한 정보를 확보했습니다. Krokiet(현재 권장 GUI)과 CLI(czkawka_cli) 사용법을 자세히 정리해 드리겠습니다.

1. Krokiet GUI 사용법

Krokiet은 현재 공식적으로 권장되는 최신 GUI(구 GTK GUI 대체)입니다.

화면 구성 (총 6 개 영역)

영역	설명
① 왼쪽 패널	도구 선택 목록(14개 도구), 맨 아래 톱니바퀴(설정) 아이콘
② 결과 영역	스캔 결과가 그룹별로 표시되는 메인 영역
③ 액션 바	결과 영역 아래. Scan/Stop 버튼, 선택 도우미, 삭제·이동 등 액션 버튼
④ 상태 바	액션 바 바로 아래. 스캔 상태나 결과 요약("N개 항목을 Xms 만에 발견") 표시
⑤ 디렉토리 패널	폴더 아이콘으로 토글. 포함 경로(왼쪽)/제외 경로(오른쪽) 관리
⑥ 오른쪽 패널	이미지 미리보기 또는 도구별 세부 설정(옵션에 따라 표시)

사용 가능한 14 개 도구

Duplicate Files(중복 파일), Empty Folders(빈 폴더), Big Files(큰 파일), Empty Files(빈 파일), Temporary Files(임시 파일), Similar Images(유사 이미지), Similar Videos(유사 동영상), Same Music(동일 음악), Invalid Symlinks(깨진 심볼릭 링크), Broken Files(손상된 파일), Bad Extensions(잘못된 확장자), Bad Names(문제 있는 파일명), Exif Remover(EXIF 제거), Video Optimizer(동영상 최적화)

기본 사용 흐름

1. 왼쪽 패널에서 원하는 도구(예: Duplicate Files) 클릭
2. 폴더 아이콘을 눌러 디렉토리 패널 열기 → Included paths 에 스캔할 폴더 추가 (필요 시 Excluded paths 도 추가)
 - 특정 폴더를 삭제로부터 보호하려면 Ref(Reference) 체크박스 체크 → 참조 경로는 자동 삭제/이동 대상에서 제외됨
1. 오른쪽 패널에서 도구별 세부 옵션 설정 (아래 상세 참고)
2. Scan 버튼 클릭
3. 결과가 나오면:
 - Select 버튼: Select All / Unselect All / 가장 오래된-최신-큰-작은 파일만 선택(또는 제외) / 경로 최단-최장 선택 / 선택 반전 / 사용자 정의 필터 등
 - 행을 좌클릭하면 선택 토글, Ctrl+클릭은 개별 토글, Shift+클릭은 범위 선택
 - 더블클릭으로 파일을 기본 프로그램으로 열기
 - 우클릭 컨텍스트 메뉴: 파일 열기, 상위 폴더 열기, 결과에서 제거, 폴더 전체 결과 제거, 이름/경로 복사, 이름 변경 등
1. Move(이동) / Delete(영구 삭제) / Trash(휴지통으로 이동, 복구 가능) / Save(결과 파일로 저장)

주요 도구별 세부 옵션

중복 파일(Duplicate Files)

- 검사 방식: Hash(기본, 권장) / Size / Name / Size and Name
- 해시 종류: Blake3(기본) / CRC32 / XXH3

유사 이미지(Similar Images)

- Hash size: 8/16/32/64 (기본 16, 클수록 정밀)
- Resize 알고리즘: Lanczos3(기본) 등
- Hash type: Mean(기본)/Gradient/BlockHash 등
- Max difference: 기본 10 (클수록 더 많이 매칭, 느슨한 기준)
- Geometric invariance: 미러/회전된 이미지도 감지 가능

유사 동영상(Similar Videos) — ffmpeg 설치 필요

- 비주얼 비교 모드 / 오디오 지문(fingerprint) 비교 모드

- 프리셋: Near-identical / Similar / Movies 등

EXIF Remover

- 보존할 태그를 콤마로 지정 가능, 비워두면 전체 태그 제거 → Clean 버튼으로 실행

Video Optimizer

- Crop(블랙바 제거) 또는 Transcode(코덱 변환) 모드 → Optimize 버튼

설정(Settings, 톱니바퀴 아이콘)

- 스캔 필터: 허용/제외 확장자, 제외 패턴, 파일 크기(KB), 재귀 검색 여부
- 일반: 언어, 다크 테마, 아이콘만 표시, 하드링크 숨기기
- 성능: 스레드 수(재시작 필요), 캐시 사용 여부, prehash 사용, 오래된 캐시 자동 삭제
- 프리셋: 여러 스캔 설정 조합을 저장/전환 가능

실전 예시 — 백업 폴더를 보호하며 중복 제거

1. 작업 폴더 + 백업 폴더를 Included paths 에 추가
2. 백업 폴더 행에서 Ref 체크
3. Duplicate Files 도구, Hash 방식, Blake3 선택 → Scan
4. Select → "Except newest" 등 선택
5. 참조 경로 파일이 선택 안 됐는지 확인 후 Trash/Delete

2. CLI (czkawka_cli) 사용법

기본 사용법

```
czkawka_cli --help          # 전체 명령어 목록
czkawka_cli dup --help      # 특정 도구(dup)의 상세 옵션 확인
```

주요 서브커맨드

명령어	기능
dup	중복 파일 찾기
empty-folders	빈 폴더 찾기
big	가장 큰(작은) 파일 찾기
empty-files	빈 파일 찾기
temp	임시 파일 찾기
image	유사 이미지 찾기

video	유사 동영상 찾기
music	동일/유사 음악 찾기
symlinks	깨진 심볼릭 링크 찾기
broken	손상된 파일 찾기

공통 옵션 예시

- -d : 검사할 디렉토리 (여러 개 지정 가능)
- -e : 제외할 디렉토리
- -x : 제외할 확장자 (또는 IMAGE, VIDEO 같은 키워드 사용 가능)
- -f : 결과를 저장할 파일
- -D : 검사 후 삭제 옵션 조합 (예: aeo)
- -R : 재귀 검색
- -n : (big 명령어) 찾을 파일 개수
- -m : (dup 명령어) 최소 파일 크기
- -s : (dup 명령어) 검사 방식 (hash 등)
- -E : 제외 패턴 (glob)
- -z : (music 명령어) 비교할 태그 목록

실전 예시

1) 중복 파일 찾기

```
czkawka_cli dup -d /home/user -e /home/user/Pictures -m 25 -x 7z rar
IMAGE -s hash -f results.txt -D aeo
```

→ /home/user를 스캔하되 /home/user/Pictures는 제외, 25KB 이상 파일, 7z/rar/이미지 파일 제외, hash 방식, 결과를 results.txt에 저장, 삭제 옵션 적용

2) 빈 폴더 찾기

```
czkawka_cli empty-folders -d /home/user/rr /home/gateway -f results.txt
```

3) 가장 큰 파일 25개 찾기

```
czkawka_cli big -d /home/user/ /home/other -e /home/user/exclude -n 25 -x
VIDEO -f results.txt
```

4) 빈 파일 찾기 (재귀)

```
czkawka_cli empty-files -d /home/user /home/other -e /home/user/Desktop -
R -f results.txt
```

5) 임시 파일 찾기 후 삭제

```
czkawka_cli temp -d /home/user/ -E */.git */tmp* *Desktop -f results.txt -D
```

6) 동일 음악 찾기

```
czkawka_cli music -d /home/user -e /home/user/Desktop -z "artist,year,ARTISTALBUM,ALBUM__tiTlE" -f results.txt
```

7) 깨진 심볼릭 링크 찾기

```
czkawka_cli symlinks -d /home/user/ /home/other -e /home/user/exclude -x jpg -f results.txt
```

8) GUI 미리보기 속도 향상용 — 동영상 썸네일 사전 생성

```
czkawka_cli video -d /your/videos --generate-thumbnails
```

→ 이 캐시를 Krokiet GUI가 그대로 활용해 미리보기 로딩이 빨라집니다.

CLI 참고 사항

- 유사 동영상 검색 기능은 ffmpeg 가 별도로 설치되어 있어야 동작합니다.
- HEIF/RAW/AVIF 등 확장 이미지 포맷은 사전 빌드된 CLI 바이너리에 기본 포함되지 않는 경우가 많으니, 필요 시 heif,libraw,libavif 기능을 활성화해 직접 빌드해야 합니다.
- 캐시와 설정 파일은 GUI(Krokiet)와 공유됩니다 — Linux 기준 캐시는 ~/.cache/czkawka/에 저장됩니다.

궁금하신 특정 도구(예: 유사 이미지 세부 옵션, 동영상 최적화 등)나 자동화 스크립트가 필요하시면 말씀해 주세요.

5. 사용자

유사 이미지 세부옵션과 자동화 스크립트가 어떻게 있는지 알려줘

6. Claude

유사 이미지(Similar Images) 세부 옵션

GUI(Krokiet) 세부 옵션 — 오른쪽 패널

옵션	기본값	설명
Hash size	16	8 / 16 / 32 / 64 중 선택. 클수록 더 정밀하게 비교하지만, 값이 클수록 Max difference 임계값도 함께 올려야 함
Resize 알고리즘	Lanczos3	Lanczos3(고품질·느림) / Nearest(빠름·저품질) / Triangle / Gaussian / CatmullRom
Hash type	Mean	Mean(빠름) / Gradient(균형) / BlockHash(리사이즈 없이 비교, 매우 유사한 이미지에 적합) / VertGradient / DoubleGradient / Median(색상 변화에 강함)
Geometric invariance	Off	Off / Mirror+Flip(좌우반전 감지) / Mirror+Flip+Rotate90(회전까지 감지)
Ignore images with same size	Off	모든 이미지의 바이트 크기가 동일한 그룹은 건너뛴
Ignore images with same resolution	Off	모든 이미지의 픽셀 해상도가 동일한 그룹은 건너뛴
Max difference	10	해시 간 최대 허용 차이. 클수록 더 많이 매칭(느슨), 작을수록 엄격

전역 설정(Settings → Similar Images tool 섹션)

- Image preview: 선택한 이미지의 썸네일 표시 여부

CLI(czkawka_cli image) 세부 옵션

```
czkawka_cli image --help
```


플래그	기본값	설명
-d, --directories	(필수)	검사할 디렉토리 (복수 지정 가능)
-e, --excluded-directories		제외 디렉토리
-E, --excluded-items		와일드카드 제외 패턴
-x, --allowed-extensions		허용 확장자 (기본 IMAGE 키워드 사용 가능)
-s, --minimal-file-size	16384 (16KB)	최소 파일 크기(bytes)
-i, --maximal-file-size	무제한	최대 파일 크기(bytes)
-s, --max-difference	5	최대 차이(0~40). hash_size 8일 땐 10 이하, 16일 땐 20 이하 권장
-g, --hash-alg	Gradient	Mean / Gradient / Blockhash / VertGradient / DoubleGradient / Median
-z, --image-filter	Nearest	Lanczos3 / Nearest / Triangle / Gaussian / CatmullRom
-c, --hash-size	16	8 / 16 / 32 / 64
--geometric-invariance	off	off / mirror-flip / mirror-flip-rotate90
--ignore-same-size		동일 크기 이미지 그룹 무시
--ignore-same-resolution		동일 해상도 이미지 그룹 무시
-r, --reference-directories		참조(보호) 디렉토리 지정
-D, --delete-method		삭제 방식 지정 (자세한 옵션 값은 --help 참조)
-f, --file-to-save		결과를 텍스트 파일로 저장
-R, --not-recursive		하위 폴더 검색 안 함
-T, --thread-number	0(전체)	사용할 스레드 수
-H, --disable-cache		캐시 비활성화(느려지지만 항상 최신 결과)

예시

```
czkawka_cli image -d /home/user/Pictures -x IMAGE -c 16 -g Gradient -s 10  
-f similar_images.txt
```

→ Pictures 폴더에서 hash size 16, Gradient 알고리즘, 최대 차이 10으로 유사 이미지 검색 후 결과 저장

자동화 스크립트 예시

1. Linux/Mac — Bash 스크립트 (중복 파일 자동 정리 + 로그)

```
#!/bin/bash
# duplicate_cleanup.sh
TARGET_DIR="$HOME/Downloads"
BACKUP_REF="$HOME/Backup"      # 보호할 참조 폴더
RESULT_FILE="$HOME/czkawka_logs/dup_$(date +%Y%m%d_%H%M%S).txt"

mkdir -p "$HOME/czkawka_logs"

czkawka_cli dup \
  -d "$TARGET_DIR" "$BACKUP_REF" \
  -r "$BACKUP_REF" \
  -m 1024 \
  -s hash \
  -f "$RESULT_FILE"

echo "완료: 결과는 $RESULT_FILE 에 저장되었습니다."
```

cron으로 매주 실행 등록:

```
crontab -e
# 매주 일요일 새벽 3시 실행
0 3 * * 0 /home/user/scripts/duplicate_cleanup.sh
```

2. Windows — 배치 스크립트(.bat)

```
@echo off
set TARGET_DIR=C:\Users\%USERNAME%\Downloads
set RESULT_DIR=C:\Users\%USERNAME%\czkawka_logs
if not exist "%RESULT_DIR%" mkdir "%RESULT_DIR%"

czkawka_cli.exe dup -d "%TARGET_DIR%" -m 1024 -s hash -f
"%RESULT_DIR%\dup_result.txt"

echo 완료. 결과 파일: %RESULT_DIR%\dup_result.txt
pause
```

작업 스케줄러(Task Scheduler)에 등록해서 정기 실행 가능합니다.

3. 유사 이미지 자동 스캔 + JSON 저장 (분석용)

```
#!/bin/bash
czkawka_cli image \
  -d "$HOME/Pictures" \
  -c 16 -g Gradient -s 10 \
  --json-compact-file-to-save "$HOME/czkawka_logs/similar_images.json"
```

JSON으로 저장하면 이후 Python 등으로 결과를 파싱해서 추가 처리(예: 자동 삭제 후보 필터링)를 할 수 있습니다.

4. GUI 미리보기 속도 향상용 — 동영상 썸네일 사전 생성(정기 배치)

```
#!/bin/bash
# 매일 밤 새 동영상의 썸네일 캐시를 미리 만들어 둬
czkawka_cli video -d "$HOME/Videos" --generate-thumbnails
```

5. 여러 도구를 순차 실행하는 통합 정리 스크립트

```
#!/bin/bash
LOG_DIR="$HOME/czkawka_logs"
mkdir -p "$LOG_DIR"
DATE=$(date +%Y%m%d)

echo "1. 빈 폴더 검사..."
czkawka_cli empty-folders -d "$HOME/Documents" -f
"$LOG_DIR/empty_folders_$DATE.txt"

echo "2. 임시 파일 검사..."
czkawka_cli temp -d "$HOME/Documents" -f "$LOG_DIR/temp_$DATE.txt"

echo "3. 중복 파일 검사..."
czkawka_cli dup -d "$HOME/Documents" -m 1024 -s hash -f
"$LOG_DIR/dup_$DATE.txt"

echo "4. 유사 이미지 검사..."
czkawka_cli image -d "$HOME/Pictures" -c 16 -g Gradient -s 10 -f
"$LOG_DIR/images_$DATE.txt"

echo "모든 검사 완료. 로그: $LOG_DIR"
```

참고

- CLI 는 스캔만 하고 실제 삭제는 별도 플래그(-D, -F, --fix-* 등 도구별로 다름)가 있어야 실행됩니다. 자동 삭제 스크립트를 만드실 경우 먼저 결과 파일만 저장해서 검토한 뒤 삭제 단계를 추가하는 것을 권장드립니다.

- 정확한 삭제 옵션 값(-D 뒤에 오는 문자 조합 등)은 버전마다 조금씩 다를 수 있으니, 실행 전 `czkawka_cli dup --help` 로 최신 옵션을 확인하시는 것이 안전합니다.

7. 사용자

czkawka가 wsl의 fdupes를 대체하는 더 좋은 툴인데 이런거처럼 wsl의 툴을 대체할 수 있는 더 좋은 툴들을 다 알려줘

8. Claude

WSL(리눅스) 환경에서 fdupes → czkawka처럼, 고전 유닉스 도구들을 대체하는 현대적(주로 Rust 기반) 도구들이 최근 몇 년간 정말 많이 나왔습니다. 카테고리별로 정리해 드립니다.

파일 탐색 / 목록

기존 도구	대체 도구	특징
ls	eza (구 exa 후속)	색상, 아이콘, Git 상태 표시, 트리 뷰
ls	lsd	eza와 유사, ls와 완전 호환
find	fd	훨씬 빠름, .gitignore 자동 인식, 직관적 문법
cd	zoxide	방문 빈도/최근성 기반 스마트 디렉토리 이동 (z 폴더명)
tree	broot	대화형 트리 탐색, 검색·필터 내장
파일 관리자	yazi	터미널용 비동기 파일 매니저, 이미지/영상 미리보기

텍스트 검색 / 편집

기존 도구	대체 도구	특징
grep	ripgrep (rg)	압도적으로 빠름, gitignore 인식
grep (문서 포함)	ripgrep-all (rga)	PDF, Office, 압축파일 내부까지 검색
cat	bat	문법 하이라이팅, Git diff 표시, 자동 페이징
sed	sd	훨씬 직관적인 정규식 치환 문법
cut/awk	choose, hck	더 사람 친화적인 컬럼 추출
diff	delta	git/diff 결과를 보기 좋게 렌더링
jq(JSON) 계열	jq(원조), 확장으로 jless	JSON 대화형 뷰어

디스크 / 시스템 모니터링

기존 도구	대체 도구	특징
du	dust	시각적 막대그래프로 용량 큰 폴더 즉시 확인
df	duf	보기 좋은 표 형태 디스크 사용량
top/htop	bottom (btm), ytop	그래프 기반 실시간 리소스 모니터
ps	procs	컬러, 트리 뷰, 검색 지원
dmesg	rmesg	Rust 기반 커널 로그 뷰어
네트워크 사용량	bandwhich	프로세스별 대역폭 실시간 표시

파일 중복/정리 (요청하신 czkawka 계열)

기존 도구	대체 도구	특징
fdupes	czkawka / Krokiet	GUI+CLI, 중복뿐 아니라 유사이미지/영상/음악까지
fdupes (CLI 전용 대안)	fclones	Rust로 작성된 매우 빠른 중복 탐색기, 하드링크/심볼릭링크 자동화
rm (안전한 삭제)	rip (rm-improved)	휴지통 방식 삭제, 실수로 지운 파일

		복구 가능
파일 이름 일괄 변경	rnre	배치 리네임 도구

Git 관련

기존 도구	대체 도구	특징
git CLI 직접 조작	lazygit	TUI 기반, 스테이징/커밋/리베이스를 키보드로 시각적 처리
git diff	delta	위 표와 동일, git pager로 설정 가능
커밋 시크릿 검사	ripsecrets	코드에 API 키 등 실수로 커밋되는 것 방지

셸 히스토리 / 생산성

기존 도구	대체 도구	특징
history	atuin	SQLite 기반 히스토리, 디렉토리/세션별 필터, 기기 간 암호화 동기화
man/tldr	tealdeer	Rust 기반 tldr 클라이언트, 오프라인 캐시
time	hyperfine	통계적으로 신뢰도 높은 벤치마킹(반복 실행, 평균/표준편차)
프롬프트 커스터마이징	starship	매우 빠른 크로스셸 프롬프트
bash/zsh 자체	nushell	구조화된 데이터(표) 기반의 새로운 셸
Ctrl+R 검색	fzf	범용 퍼지 파인더, 히스토리/파일/git 브랜치 검색 등에 조합 활용

코드 통계 / 개발 보조

기존 도구	대체 도구	특징
wc -l (코드 라인 수)	tokei	언어별 코드 라인/공백/주석 통계
find + 파일 변경 감지	watchexec	파일 변경 시 명령 자동 실행
CSV 처리	qsv (xsv 후속)	매우 빠른 CSV 슬라이싱/분석

```
# cargo 설치되어 있다면
cargo install ripgrep fd-find bat eza zoxide git-delta hyperfine du-dust
bottom starship tokei tealdeer fclones rnr

# 또는 apt로 가능한 것들 (버전은 다소 구버전일 수 있음)
sudo apt install ripgrep fd-find bat

# fzf, lazygit, atuin, yazi 등은 각자 공식 설치 스크립트/GitHub Release
사용 권장
```

참고: 신중하게 접근할 부분

- 인터랙티브 사용 vs 스크립트: rg, fd, bat 등은 대화형 작업에는 훌륭하지만, 다른 사람과 공유하는 셸 스크립트나 CI 파이프라인에서는 여전히 표준 grep/find/cat 을 쓰는 것이 이식성 면에서 안전합니다(모든 서버에 Rust 도구가 깔려 있는 건 아니므로).
- czkawka/fclones 처럼 완전히 새로운 카테고리의 도구는 대체라기보다 "상위 호환"에 가깝습니다 — GD RDM 처럼 대량 CSV·중복 데이터를 다루는 작업 환경이시라면 fclones 나 czkawka_cli 를 스크립트에 넣어 정기 정리 자동화하는 것도 유용할 수 있습니다.

특정 카테고리(예: 파일 검색 도구들 조합 workflow, 또는 GD RDM 프로젝트에 맞는 CSV 처리 도구)를 더 깊이 알고 싶으시면 말씀해 주세요.

9. 사용자

파일 검색 도구들 조합 workflow

10. Claude

파일 검색 도구 조합 워크플로우

fd + ripgrep + fzf + bat + zoxide를 조합하면 "파일 찾기 → 내용 검색 → 미리보기 → 열기"까지 한 번에 이어지는 강력한 검색 파이프라인을 만들 수 있습니다. 각 도구의 역할부터 실전 조합까지 정리해 드립니다.

1. 각 도구의 역할 분담

도구	역할	대체하는 것
fd	파일명/경로로 찾기	find
ripgrep (rg)	파일 내용으로 찾기	grep
fzf	목록을 대화형으로 필터링/선택	파이프라인의 "허브" 역할
bat	미리보기(문법 하이라이팅)	cat, less
zoxide	자주 가는 폴더로 즉시 이동	cd

핵심 아이디어는: fd나 rg가 후보를 뽑아내고 → fzf가 그 목록을 실시간 필터링하며 미리보기를 보여주고 → 선택된 결과를 편집기/명령으로 넘긴다는 파이프라인 구조입니다.

2. 기본 조합 예시

① 파일명으로 찾고, fzf로 골라서 bat로 미리보기

```
fd . --type f | fzf --preview 'bat --color=always --style=numbers {}'
```

② 파일 내용으로 검색하고, 매칭된 줄 주변까지 미리보기

```
rg --line-number --no-heading --color=always "검색어" \
| fzf --ansi \
    --delimiter ':' \
    --preview 'bat --color=always --highlight-line {2} {1}'
```

→ rg가 파일:줄번호:내용 형태로 출력 → fzf가 그걸 파싱해서 해당 줄을 하이라이트한 미리보기 제공

③ 선택한 파일을 바로 편집기(VS Code / vim)로 열기

```
rg --line-number --no-heading --color=always "TODO" \
| fzf --ansi --delimiter ':' \
    --preview 'bat --color=always --highlight-line {2} {1}' \
| awk -F: '{print $1 " " $2}' \
| xargs -r code -g
```

3. 함수로 만들어서 .bashrc/.zshrc 에 등록

fzf + rg 통합 검색 함수 (내용 검색용)

```
# 사용법: rgf "검색어"
rgf() {
    local file line
    read -r file line <<< "$(rg --line-number --no-heading --color=always
"$1" \
    | fzf --ansi --delimiter ':' \
        --preview 'bat --color=always --highlight-line {2} {1}' \
    | awk -F: '{print $1, $2}')"
    [ -n "$file" ] && ${EDITOR:-vim} +"$line" "$file"
}
```

fzf + fd 통합 함수 (파일명 검색 후 이동/열기)

사용법: ff (파일 선택 후 vim으로 열기)

```
ff() {
    local file
    file=$(fd --type f --hidden --exclude .git | fzf --preview 'bat --
color=always --style=numbers {}')
    [ -n "$file" ] && ${EDITOR:-vim} "$file"
}
```

사용법: fcd (폴더 선택 후 이동)

```
fcd() {
    local dir
    dir=$(fd --type d --hidden --exclude .git | fzf --preview 'eza --tree -
-level=2 --color=always {}')
    [ -n "$dir" ] && cd "$dir"
}
```

4. zoxide + fzf 결합 — "최근/자주 쓴 폴더 중에서 fzf 로 선택"

zoxide가 기억한 폴더 목록을 fzf로 필터링해서 이동

```
zi() {
    local dir
    dir=$(zoxide query -l | fzf --preview 'eza --tree --level=2 --
color=always {}')
    [ -n "$dir" ] && cd "$dir"
}
```

참고: zoxide 자체에도 zi 명령이 내장되어 있어서, zoxide init 설정만 해두면 별도 함수 없이 바로 zi로 fzf 연동 이동이 됩니다.

5. Git 브랜치/커밋 검색까지 확장

Git 브랜치를 fzf로 선택 후 체크아웃

```
gco() {
    local branch
    branch=$(git branch -a | grep -v HEAD | sed 's/^[* ]*//' | fzf --
preview 'git log --oneline --graph --color=always {}')
```

```
[ -n "$branch" ] && git checkout "${branch#remotes/origin/}"
}
```

6. Ctrl+R / Ctrl+T 단축키 (fzf 기본 제공)

fzf를 셸에 설치하면(fzf --bash 또는 ~/.fzf.bash) 기본적으로 다음이 자동 활성화됩니다.

단축키	기능
Ctrl+T	현재 위치 기준 파일/폴더를 fzf로 찾아 커맨드라인에 삽입
Ctrl+R	셸 히스토리를 fzf로 검색 (atuin 사용 시 더 강력)
Alt+C	fzf로 폴더 선택 후 즉시 cd

fd를 fzf의 기본 검색 엔진으로 바꾸면 훨씬 빨라집니다.

```
export FZF_DEFAULT_COMMAND='fd --type f --hidden --exclude .git'
export FZF_CTRL_T_COMMAND="$FZF_DEFAULT_COMMAND"
export FZF_CTRL_T_OPTS="--preview 'bat --color=always --style=numbers {}'"
export FZF_ALT_C_COMMAND='fd --type d --hidden --exclude .git'
export FZF_ALT_C_OPTS="--preview 'eza --tree --level=2 --color=always {}'"
```

7. 실전 시나리오 예시

시나리오 A — 코드베이스에서 특정 함수 정의 찾아 바로 편집

```
rgf "def process_data"
```

→ 매칭 목록이 뜨고, 화살표로 고르면 해당 줄이 하이라이트된 미리보기가 뜨고, Enter 시 편집기가 그 줄로 바로 열림

시나리오 B — 대형 프로젝트(예: GD RDM CSV 스키마 작업)에서 특정 컬럼명이 들어간 스크립트 찾기

```
rg "diagnostic_code" -t py -t sql | fzf --ansi --delimiter ':' --preview 'bat --highlight-line {2} {1}'
```

시나리오 C — 여러 프로젝트 폴더를 오가며 작업할 때

```
zi # zoxide + fzf로 최근 작업 폴더 중 선택 → 이동
ff # fd + fzf로 그 폴더 안 파일 선택 → 편집기 오픈
```

참고: WSL 에서 설치

```
sudo apt update
sudo apt install fd-find ripgrep bat fzf
# Ubuntu에서는 fd, bat 실행파일명이 fdfind, batcat으로 설치되는 경우가 많음
→ alias 필요
echo 'alias fd=fdfind' >> ~/.bashrc
echo 'alias bat=batcat' >> ~/.bashrc

# zoxide는 apt 버전이 낮은 경우가 많아 공식 스크립트 권장
curl -sS
https://raw.githubusercontent.com/ajeetdsouza/zoxide/main/install.sh |
bash
```

원하시면 위 함수들을 최상길 님의 .bashrc/.zshrc에 바로 넣을 수 있는 완성된 설정 파일(dotfile)로 만들어 드릴 수도 있습니다. 필요하신가요?

11. 사용자

만들어 줘

12. Claude

적용 방법은 다음과 같습니다.

```
# 1) 원하는 위치로 이동/저장 (예시)
mkdir -p ~/.config
mv search_tools.sh ~/.config/search_tools.sh

# 2) ~/.bashrc 맨 아래에 추가
echo 'source ~/.config/search_tools.sh' >> ~/.bashrc

# 3) 적용
source ~/.bashrc
```

포함된 기능

- ff — 파일명 검색 → 미리보기 → 편집기로 열기
- fcd — 폴더명 검색 → 이동
- rgf "검색어" — 파일 내용 검색 → 해당 줄 미리보기 → 편집기로 열기
- zi — zoxide 기록 중 선택 → 이동 (zoxide 내장 zi 가 있으면 그걸 우선 사용)
- gco — git 브랜치 선택 → 체크아웃
- Ctrl+T / Ctrl+R / Alt+C — fzf 기본 단축키가 fd 기반으로 동작하도록 설정
- WSL/Ubuntu 의 fdfind, batcat 이름 문제 자동 보정
- ls, cat 별칭을 eza, bat 으로 교체 (설치돼 있을 때만)

도구가 아직 설치 안 되어 있다면 이전에 안내드린 설치 명령어(sudo apt install fd-find ripgrep bat fzf + zoxide 스크립트 설치)를 먼저 실행하시면 됩니다.

[생성된 파일: search_tools.sh 전체 소스 코드]

```
#!/usr/bin/env bash
# =====
# 파일 검색 도구 조합 워크플로우 설정
# fd + ripgrep + fzf + bat + zoxide
#
# 사용법:
# 1) 이 파일을 원하는 위치에 저장 (예: ~/.config/search_tools.sh)
# 2) ~/.bashrc (또는 ~/.zshrc) 맨 아래에 아래 한 줄 추가:
#    source ~/.config/search_tools.sh
# 3) 새 터미널을 열거나 `source ~/.bashrc` 실행
#
# bash / zsh 모두에서 동작합니다.
# =====

# -----
# 0. WSL/Ubuntu 환경 alias 보정
#   Ubuntu apt 패키지는 fd, bat이 각각 fdfind, batcat 이름으로 설치됨
# -----
if command -v fdfind >/dev/null 2>&1 && ! command -v fd >/dev/null 2>&1; then
    alias fd='fdfind'
fi
if command -v batcat >/dev/null 2>&1 && ! command -v bat >/dev/null 2>&1; then
    alias bat='batcat'
fi

# -----
# 1. fzf 기본 검색 엔진을 fd로 교체 (Ctrl+T, Alt+C, 기본 fzf 호출에 적용)
# -----
if command -v fd >/dev/null 2>&1; then
    export FZF_DEFAULT_COMMAND='fd --type f --hidden --exclude .git'
    export FZF_CTRL_T_COMMAND="$FZF_DEFAULT_COMMAND"
    export FZF_ALT_C_COMMAND='fd --type d --hidden --exclude .git'
fi

if command -v bat >/dev/null 2>&1; then
    export FZF_CTRL_T_OPTS="--preview 'bat --color=always --style=numbers {}'"
fi

if command -v eza >/dev/null 2>&1; then
    export FZF_ALT_C_OPTS="--preview 'eza --tree --level=2 --color=always {}'"
fi

# fzf 셸 통합 로드 (Ctrl+T, Ctrl+R, Alt+C 활성화)
# 설치 방식에 따라 경로가 다를 수 있어 여러 후보를 순서대로 확인
if command -v fzf >/dev/null 2>&1; then
    if [ -n "$BASH_VERSION" ]; then
        eval "$(fzf --bash 2>/dev/null)" || {
            [ -f ~/.fzf.bash ] && source ~/.fzf.bash
        }
    elif [ -n "$ZSH_VERSION" ]; then
        eval "$(fzf --zsh 2>/dev/null)" || {
            [ -f ~/.fzf.zsh ] && source ~/.fzf.zsh
        }
    fi
fi
```

```

    fi
fi

# -----
# 2. zoxide 초기화 (cd 대체 + zi 대화형 이동)
# -----
if command -v zoxide >/dev/null 2>&1; then
    if [ -n "$BASH_VERSION" ]; then
        eval "$(zoxide init bash)"
    elif [ -n "$ZSH_VERSION" ]; then
        eval "$(zoxide init zsh)"
    fi
fi

# -----
# 3. 함수: ff - 파일명으로 찾아서 미리보기 후 편집기로 열기
#   사용법: ff
# -----
ff() {
    local file
    file=$(fd --type f --hidden --exclude .git 2>/dev/null \
        | fzf --preview 'bat --color=always --style=numbers {} 2>/dev/null || cat {}') \
        || return
    [ -n "$file" ] && "${EDITOR:-vim}" "$file"
}

# -----
# 4. 함수: fcd - 폴더명으로 찾아서 이동
#   사용법: fcd
# -----
fcd() {
    local dir
    dir=$(fd --type d --hidden --exclude .git 2>/dev/null \
        | fzf --preview 'eza --tree --level=2 --color=always {} 2>/dev/null || ls -la {}') \
        || return
    [ -n "$dir" ] && cd "$dir" || return
}

# -----
# 5. 함수: rgf - 파일 내용 검색 후 해당 줄로 편집기 열기
#   사용법: rgf "검색어"
# -----
rgf() {
    if [ -z "$1" ]; then
        echo "사용법: rgf <검색어>"
        return 1
    fi

    local result file line
    result=$(rg --line-number --no-heading --color=always "$1" 2>/dev/null \
        | fzf --ansi \
            --delimiter ':' \
            --preview 'bat --color=always --highlight-line {2} {1} 2>/dev/null || cat {1}') \
        || return

    [ -z "$result" ] && return

```

```

file=$(echo "$result" | cut -d: -f1)
line=$(echo "$result" | cut -d: -f2)

[ -n "$file" ] && "${EDITOR:-vim}" "+$line" "$file"
}

# -----
# 6. 함수: zi - zoxide 기록 중에서 fzf로 골라 이동
#   (zoxide 자체 내장 zi와 겹치므로, 내장 버전이 없을 때만 정의)
# -----
if ! command -v __zoxide_zi >/dev/null 2>&1 && ! type zi >/dev/null 2>&1; then
    zi() {
        local dir
        dir=$(zoxide query -l 2>/dev/null \
            | fzf --preview 'eza --tree --level=2 --color=always {} 2>/dev/null || ls
-1a {}') \
            || return
        [ -n "$dir" ] && cd "$dir" || return
    }
fi

# -----
# 7. 함수: gco - git 브랜치를 fzf로 골라 체크아웃
#   사용법: gco
# -----
gco() {
    local branch
    branch=$(git branch -a 2>/dev/null | grep -v HEAD | sed 's/^[* ]*///' \
        | fzf --preview 'git log --oneline --graph --color=always {} 2>/dev/null |
head -100') \
        || return
    [ -z "$branch" ] && return
    git checkout "${branch#remotes/origin/}"
}

# -----
# 8. 별칭 정리
# -----
if command -v eza >/dev/null 2>&1; then
    alias ls='eza --icons --group-directories-first'
    alias ll='eza -l --icons --group-directories-first'
    alias lt='eza --tree --level=2 --icons'
fi

if command -v bat >/dev/null 2>&1; then
    alias cat='bat --paging=never'
fi

# =====
# 요약
# ff      : 파일명 검색 → 미리보기 → 편집기로 열기
# fcd     : 폴더명 검색 → 이동
# rgf     : 파일 내용 검색 → 해당 줄 미리보기 → 편집기로 열기
# zi      : 최근/자주 방문 폴더 중 선택 → 이동
# gco     : git 브랜치 선택 → 체크아웃
# Ctrl+T: 현재 위치 기준 fd로 파일 찾기 → 커맨드라인에 삽입
# Ctrl+R: 셸 히스토리 fzf 검색

```

```
# Alt+C : fd로 폴더 찾기 → cd
```

```
# =====
```